

Apache Spark — Standalone Cluster

Workers, Executors, Driver, Master, Failure Detection & Recovery

These notes cover the complete picture of Spark Standalone mode: the role of each component, where each one runs as a JVM, how failures are detected, what happens if the Driver or Master fails, how Master high availability works, and where to observe all of this in practice.

Contents

- Core Terms — Worker Node, Executor, Driver, Master, Application
- JVM Placement — What Runs Where
- Roles (Not a Strict Hierarchy)
- Heartbeat Monitoring Paths
- Failure Detection — Executor, Worker, Driver, Master
- What Spark Does After an Executor Fails
- Driver Failure — Impact and Recovery
- Master Failure — Impact and Recovery
- Master Recovery Modes — ZooKeeper HA vs Filesystem Recovery
- Important Configuration Settings
- Where to Observe Everything (UIs, Logs, History Server, APIs)
- Understanding 'Application' and spark-submit
- Quick Comparison Tables & One-Line Summary

1. Core Terms

- **Worker node:** a physical or virtual machine in the cluster. It provides CPU cores and memory, and belongs to the cluster, not to any single application.
- **Executor:** a JVM process launched on a worker node for one specific application. It runs task threads in parallel and holds cached and shuffle data in memory. It lives only as long as the application.
- **Worker daemon:** a background JVM process on every worker node. It launches executors as child processes and watches them.
- **Driver:** the JVM running your main program. It builds the execution plan and sends tasks directly to executors.
- **Master:** the cluster manager JVM. It tracks all workers and applications, and decides which workers should launch executors for a new application.
- **Task:** the smallest unit of work — a thread running inside an executor JVM, not a separate process.
- **Application:** one complete program submitted with `spark-submit`. It consists of exactly one driver and a set of executors, and exists only while that program runs.

***Analogy:** a worker node is like an office building — it exists regardless of any project. Executors are teams assigned to work inside it for one particular project, and leave when the project (application) ends.*

2. JVM Placement — What Runs Where

Every main Spark component is its own JVM process. Tasks are the only exception — they are threads, not processes.

Component	JVM process?	Where it runs
Master	Yes	Master node (one ACTIVE, plus standbys if HA is set up)
Worker daemon	Yes	Every worker node — one per node
Executor	Yes (one JVM per executor)	Worker node, launched by the Worker daemon as a child process
Driver	Yes	Client mode: the machine running <code>spark-submit</code> . Cluster mode: on one of the workers
Task	No — a thread	Inside an executor JVM (one thread per core)
History Server	Yes	Any chosen machine, default port 18080
ZooKeeper	Yes (Java-based)	Separate ZooKeeper cluster (used for Master HA)

Master node: [Master JVM]
Worker node 1: [Worker JVM] -- launches--> [Executor JVM: task threads]
Worker node 2: [Worker JVM] -- launches--> [Executor JVM: task threads]
Client machine: [Driver JVM] (or on a worker, in cluster mode)

- Worker daemon JVM and Executor JVM are different processes — the Worker only manages, the Executor does the work.
- One worker node can run many executor JVMs, from the same application or different applications.
- Executor JVM memory is separate from the Worker daemon's own memory.
- In PySpark, Python worker processes run alongside the executor JVM.
- If a JVM dies, everything inside it dies — an executor crash kills all its task threads plus its cached and shuffle data.

3. Roles (Not a Strict Hierarchy)

It is tempting to memorize *Master* → *Worker* → *Executor* → *Task* as a strict parent-child chain, but that is not accurate. The only real parent-child relationship is at the OS level: the **Worker daemon launches the executor process**. Everything else is communication between independent roles, not a hierarchy.

Role	Responsibility
Master	Manages cluster resources; tracks workers and applications
Worker	Provides resources; launches and watches executors
Driver	Coordinates your application; sends tasks directly to executors
Executor	Executes tasks; stores cached and shuffle data; reports to the driver
Task	Smallest unit of work, sent by the Driver (never by the Master) to an executor

Important: tasks are always sent by the **Driver**, never by the Master.

4. Heartbeat Monitoring Paths

There are two independent monitoring paths, plus one state-change notification:

```
Worker  --heartbeat--> Master    (worker alive?)
Executor --heartbeat--> Driver    (executor alive? + task metrics)
Worker  --state change-> Master   (executor process exited: FAILED/KILLED/EXITED)
Master  --notify-----> Driver   (executor / worker lost)
```

- The **Worker** → **Master** heartbeat tells the Master the worker node itself is alive.
- The **Executor** → **Driver** heartbeat is more than 'I'm alive' — it also carries task metrics.
- Heartbeats are the mechanism for detecting *hangs*. Process *crashes* are detected differently — the OS tells the Worker directly when its child process exits.

5. Failure Detection

5.1 Executor Failure

Detection differs depending on **how** the executor fails — this is a key distinction, not a single flow.

Failure type	Who notices first	Mechanism
Process crashes (OOM, kill -9, JVM crash)	Worker	Worker sees its child process exit -> tells Master -> Master tells Driver
Hangs (long GC pause, stuck)	Driver	Missing heartbeats past spark.network.timeout
Connection drops (network issue)	Driver	RPC disconnect event

Executor process crashes:

```
Worker sees child process exit
-> Worker sends ExecutorStateChanged to Master
-> Master tells Driver
-> Tasks rescheduled + replacement executor requested
```

Executor hangs:

```
No heartbeat for spark.network.timeout (120s)
-> Driver declares executor lost
-> Tasks rescheduled
```

5.2 Worker Node Failure

Worker sends no heartbeat for `spark.worker.timeout` (60s)

- > Master marks the Worker DEAD
- > All executors on that worker are marked lost
- > Driver is told (or sees its connections to them drop)
- > Running tasks on those executors are rescheduled elsewhere
- > Lost cache is recomputed from lineage; lost shuffle output triggers stage re-runs

5.3 Summary Table

Failure type	Detected by	Mechanism
Executor process exits	Worker -> Master -> Driver	Child process exit status
Executor hangs / long GC	Driver	Missing heartbeats past <code>spark.network.timeout</code>
Worker node dies	Master -> Driver	Missing Worker heartbeats (<code>spark.worker.timeout</code>)
Connection lost	Driver	RPC disconnect event

6. What Spark Does After an Executor Fails

- Running tasks on the lost executor are marked failed and resubmitted to other executors (up to `spark.task.maxFailures`, default 4, before the job is aborted).
- Cached data (from `cache()/persist()`) on that executor is lost and recomputed from lineage when needed.
- Shuffle output stored on that executor is lost too, unless the external shuffle service is enabled — Spark re-runs the map stages that produced it.
- The Master requests a replacement executor, up to `spark.deploy.maxExecutorRetries` (default 10).

7. Driver Failure — Impact and Recovery

The driver is the brain of the application, so its failure is the most serious of the four.

- The **whole application is killed**. Executors have no one to take orders from, so the Master removes them.
- All running jobs, cached data, and in-progress work are **lost**.
- Spark does **not** recover the job automatically by default.

How to handle it

- Use `--deploy-mode cluster --supervise`: the Master restarts the driver on another worker if it fails.
- For Structured Streaming, use a **checkpoint directory** so the restarted driver continues from where it stopped instead of starting over.
- For batch jobs, the restarted driver re-runs the job from the beginning — so make outputs **idempotent** (safe to repeat).

8. Master Failure — Impact and Recovery

The Master is only needed for **scheduling and resource management**, so the effect is smaller than a driver failure.

Already running applications

- They **keep running** — the driver and executors already talk to each other directly, without the Master in between.
- Tasks continue to be processed normally.

What stops working

- New applications cannot be submitted.
- New executors cannot be allocated, so a failed executor cannot be replaced while the Master is down.
- Workers cannot register, and the Master web UI is unavailable.
- Worker/executor failure handling pauses until a Master is active again.

Without high availability (HA) set up, the Master is a **single point of failure**.

9. Master Recovery Modes

Set with `spark.deploy.recoveryMode`: NONE (default, no recovery), ZOOKEEPER, FILESYSTEM, or CUSTOM.

9.1 ZooKeeper High Availability

ZooKeeper is a separate coordination service that stores small pieces of data and can run **leader election** — deciding which one process among many is the boss.

Setup

- Start 2 or more Masters on different machines, all pointing to the same ZooKeeper cluster.
- Configure `spark.deploy.zookeeper.url` and `spark.deploy.zookeeper.dir`.
- Give applications and workers all Master addresses, e.g. `spark://host1:7077,host2:7077`.

How it works

- All Masters connect to ZooKeeper, which elects one leader (ACTIVE); the rest become STANDBY.
- The active Master stores its state in ZooKeeper: registered workers, running applications, and drivers.

- Workers and applications talk only to the active Master.
- If the active Master crashes, ZooKeeper elects a new leader from the standbys.
- The new leader reads the saved state from ZooKeeper, contacts workers and applications (they re-register), then marks itself ALIVE and resumes normal work.

Recovery usually takes about **1–2 minutes**. No new applications can be scheduled during this window, but already-running applications keep going. Failover is **automatic** — no human involved.

9.2 Filesystem Recovery

The single Master writes its state to a directory on disk; if it dies, restart it and it reads the state back.

- Configure `spark.deploy.recoveryDirectory`.
- On restart, the Master reads the saved files, contacts workers and applications, which re-register, then normal operation resumes.
- **No automatic failover** — needs a manual restart, or an external tool such as `monit` or a `systemd` service.
- Only one Master, so there is downtime between failure and restart.
- Use **shared storage** (e.g. NFS) for the recovery directory, otherwise state is lost if the machine itself dies.

9.3 Comparison

Point	ZooKeeper HA	Filesystem Recovery
Number of Masters	Multiple (active + standby)	One
Failover	Automatic	Manual (or via restart tool)
Extra component needed	ZooKeeper cluster	None (just a directory)
Downtime	Short (~1-2 min)	Depends on when restart happens
Best for	Production	Development / testing / small setups
Survives machine loss	Yes	Only if directory is on shared storage

Analogy: ZooKeeper HA is a company with a manager and a deputy — a referee (ZooKeeper) instantly promotes the deputy if the manager leaves, and the deputy already has the manager's notes. Filesystem recovery is a company with one manager who keeps a notebook — if the manager leaves, work pauses until a new manager is hired, who reads the notebook and continues.

What gets recovered in both modes: the list of registered **workers**, running **applications**, and (in cluster mode) **drivers**. Running executors and tasks are not disturbed, since they talk directly to the driver.

10. Important Configuration Settings

Setting	Default	Meaning
spark.executor.heartbeatInterval	10s	How often an executor sends a heartbeat to the driver
spark.network.timeout	120s	No heartbeat for this long -> executor declared lost
spark.worker.timeout	60s	No heartbeat from worker for this long -> Master marks it DEAD
spark.task.maxFailures	4	Task failures allowed before the job is aborted
spark.deploy.maxExecutorRetries	10	Limit on replacement executor launches
spark.deploy.recoveryMode	NONE	NONE / ZOOKEEPER / FILESYSTEM / CUSTOM
spark.deploy.zookeeper.url	-	ZooKeeper connection string for Master HA
spark.deploy.zookeeper.dir	-	ZooKeeper path where Master state is stored
spark.deploy.recoveryDirectory	-	Directory used for filesystem recovery mode

Rule of thumb: the heartbeat interval must be much smaller than the network timeout.

11. Where to Observe Everything

11.1 Web UIs

- **Master UI** (<http://<master-host>:8080>) — Master status (ALIVE / STANDBY / RECOVERING), list of workers with state, running/completed applications, and drivers (cluster mode).
- **Worker UI** (<http://<worker-host>:8081>) — executors on that worker, their state (RUNNING / FAILED / KILLED / EXITED), and links to stdout/stderr logs.
- **Driver (Application) UI** (<http://<driver-host>:4040>, or 4041/4042... for multiple apps) — Executors tab (active and dead executors, memory, GC time), Jobs/Stages tab (failed tasks, retries, stage re-runs), Storage tab (cached data), Environment tab (all config values).

11.2 Log Files

Log	Location (Standalone default)	What you find
Master log	\$SPARK_HOME/logs/spark-*-Master-*.out	Worker lost, executor state changes, leader election, recovery messages
Worker log	\$SPARK_HOME/logs/spark-*-Worker-*.out	Executors launched, exit codes, failures
Executor logs	\$SPARK_HOME/work///stdout & stderr	OOM errors, exceptions, task output
Driver log	Console (client mode) or work/driver-*/ (cluster mode)	Executor lost messages, task failure summary

Typical messages to search for:

- Removing executor ... because it has been lost
- Executor lost / ExecutorLostFailure
- Removing worker ... because we got no heartbeat in 60 seconds
- Master has changed / I have been elected leader! New state: RECOVERING
- Recovery complete - resuming operations!

- Task ... failed ... exceeded maxFailures

11.3 Spark History Server

The live driver UI (port 4040) disappears once the application ends. The History Server keeps it for post-mortem analysis.

```
spark.eventLog.enabled=true
spark.eventLog.dir=hdfs:///spark-logs
spark.history.fs.logDirectory=hdfs:///spark-logs

# start it
sbin/start-history-server.sh
# open http://<host>:18080
```

11.4 REST API and Metrics

- Driver: /api/v1/applications/<app-id>/executors (add /allexecutors to include dead ones)
- History Server: http://<host>:18080/api/v1/applications
- Master: http://<master>:8080/json — cluster state, workers, and apps
- Metrics system (conf/metrics.properties) can send metrics to Prometheus, Graphite, JMX, or CSV for alerting.

11.5 ZooKeeper Side (for HA)

- Master UI shows one Master as ALIVE, others as STANDBY.
- zkCli.sh then "ls /spark" (the configured zookeeper.dir) shows the saved state.
- Master logs show election and recovery messages.

11.6 Quick Mapping

I want to see...	Look here
Which executors are dead and why	Driver UI -> Executors tab, executor stderr
Which worker is dead	Master UI (8080), Master log
Failed tasks and retries	Driver UI -> Jobs / Stages tab
Lost cache	Driver UI -> Storage tab
Which Master is active	Master UI status field
Recovery after Master failure	Master log, Master UI status (RECOVERING -> ALIVE)
Driver restart (supervise)	Master UI -> Running Drivers, Master log
Old application failure analysis	History Server (18080)
Config values like timeouts	Driver UI -> Environment tab

12. Understanding 'Application' and spark-submit

An **application** is your code *plus* the running instance of it on the cluster — the code is the source, the application is that code running, using one driver and a set of executors.

```
spark-submit --master spark://master:7077 my_job.py
```

Part	Meaning
spark-submit	The tool that sends your application to Spark (\$SPARK_HOME/bin)
--master spark://master:7077	Address of the Standalone Master (hostname + port 7077)
my_job.py	Your code file (a .jar for Scala/Java)

What happens after you run it:

- spark-submit contacts the Master at master:7077.
- The Master registers the application and gives it an ID (e.g. app-20260928120000-0001).
- The Master tells Workers to launch executors for it.
- The driver starts running your code and sends tasks to the executors.
- When the code finishes, the application ends and its executors are released.

Common options:

```
spark-submit \  
  --master spark://master:7077 \  
  --deploy-mode cluster \  
  --supervise \  
  --executor-memory 4G \  
  --executor-cores 2 \  
  --total-executor-cores 8 \  
  my_job.py
```

- One spark-submit run = one application (with one SparkSession/SparkContext).
- Two spark-submit runs = two separate applications, each with its own driver and executors, even on the same workers.
- With Master HA, list all Masters: --master spark://host1:7077,host2:7077

13. Quick Comparison & One-Line Summary

Component fails	Effect	Recovery
Executor JVM	Tasks re-run, small delay	Automatic
Worker node	All executors on it lost	Automatic re-run elsewhere
Driver JVM	Whole application dies	--supervise + checkpointing
Master JVM	Running jobs continue, no new scheduling	ZooKeeper HA or filesystem recovery

Executor failure: noticed by Worker (crash) or Driver (hang) — fixed automatically.

Worker failure: noticed by Master — executors rescheduled.

Driver failure: application dies unless supervised.

Master failure: running apps continue, but no new scheduling unless HA is set up.

Every component is a separate JVM — tasks alone are just threads inside the executor JVM.